



**Junta de Andalucía**  
Consejería de Educación y Deporte

# UNIDAD DE TRABAJO 11

## INTERFACES GRÁFICAS

—

## Módulo de Programación



This work is licensed under a [Creative Commons Attribution-NonCommercial-ShareAlike 4.0 International License](#).

Autor: Fran Gómez

**RA5** Realiza operaciones de entrada y salida de información, utilizando procedimientos específicos del lenguaje y librerías de clases

- f) Se han utilizado las herramientas del entorno de desarrollo para crear interfaces gráficos de usuario simples.
- g) Se han programado controladores de eventos.
- h) Se han escrito programas que utilicen interfaces gráficos para la entrada y salida de información.

1. Introducción
2. Librerías para realizar interfaces gráficas
  - a. ¿AWT, SWING, FX?
3. Componentes
  - a. Contenedores
  - b. Controladores
4. Eventos
5. Layout
6. Menús

## Character User Interfaces vs Graphical User Interface



Aplicaciones gráficas realizadas con Java



← → ↻ 🏠 localhost:8080/users/1

Spring React CRUD Users About frodo@local ▾

### Update User

Email  
frodo@local

Password  
Password (leave blank)

First name  
Frodo

Notes  
Ring bearer

### Users

| First Name | Last Name | Email       |              |                                                                           |
|------------|-----------|-------------|--------------|---------------------------------------------------------------------------|
| Frodo      | Baggins   | frodo@local | Ring bearer  | <input type="button" value="Edit"/> <input type="button" value="Delete"/> |
| Bilbo      | Baggins   | bilbo@local | Burglar      | <input type="button" value="Edit"/> <input type="button" value="Delete"/> |
| Gandalf    | the Grey  | gf@local    | Wizard       | <input type="button" value="Edit"/> <input type="button" value="Delete"/> |
| Legolas    | Greenleaf | lego@local  | Elf prince   | <input type="button" value="Edit"/> <input type="button" value="Delete"/> |
| Sam        | Gamgee    | sam@local   | The gardener | <input type="button" value="Edit"/> <input type="button" value="Delete"/> |

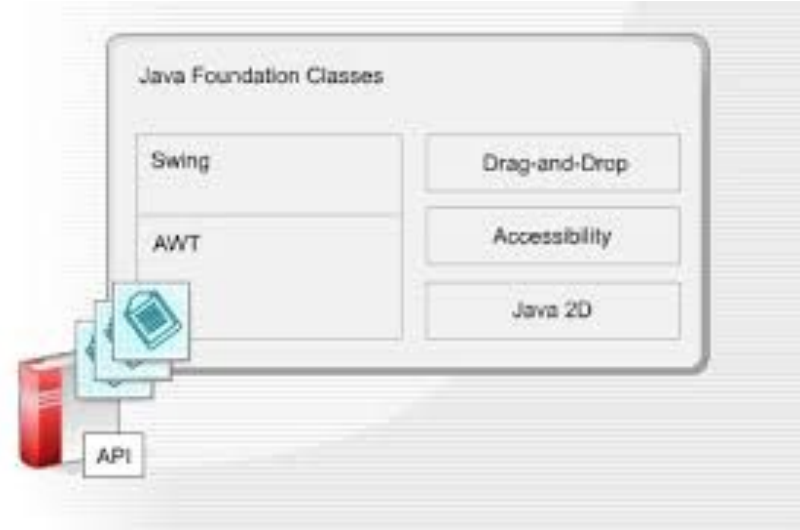
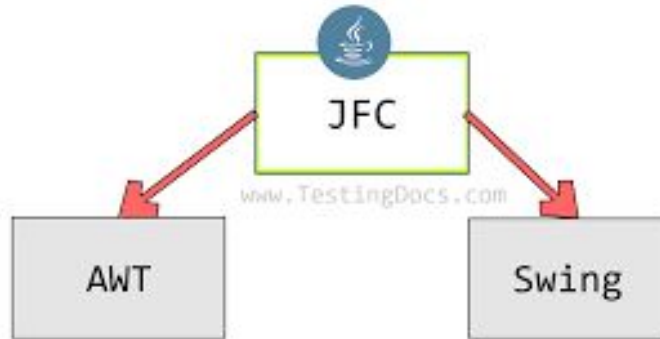
« > »

### Please sign in

Username

Password

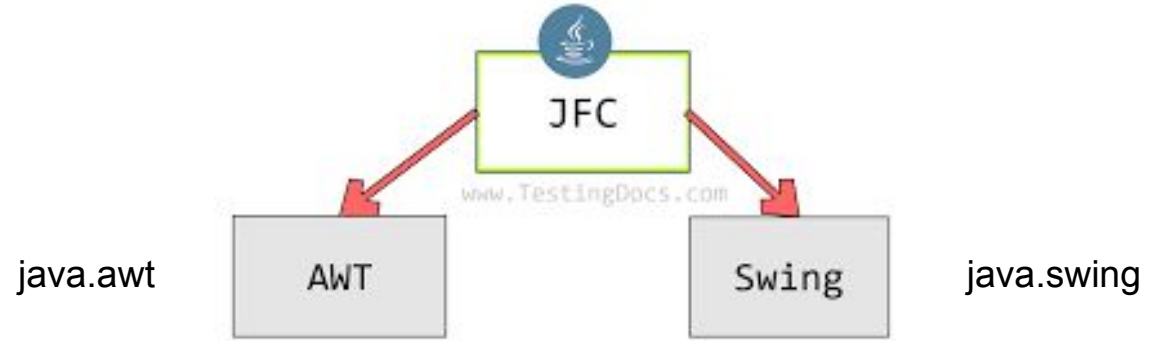
# Librerías para GUI en Java



# Librerías para GUI en Java: Conceptos previos



- Framework
- Librería
- Paquete
- Componente



<https://docs.oracle.com/javase/8/docs/api/java/awt/package-frame.html>

# Your wooclap poll will be displayed here



Install the **Chrome** or  
**Firefox extension**



Make sure you are in  
**presentation mode**

**wooclap**

# Your wooclap poll will be displayed here



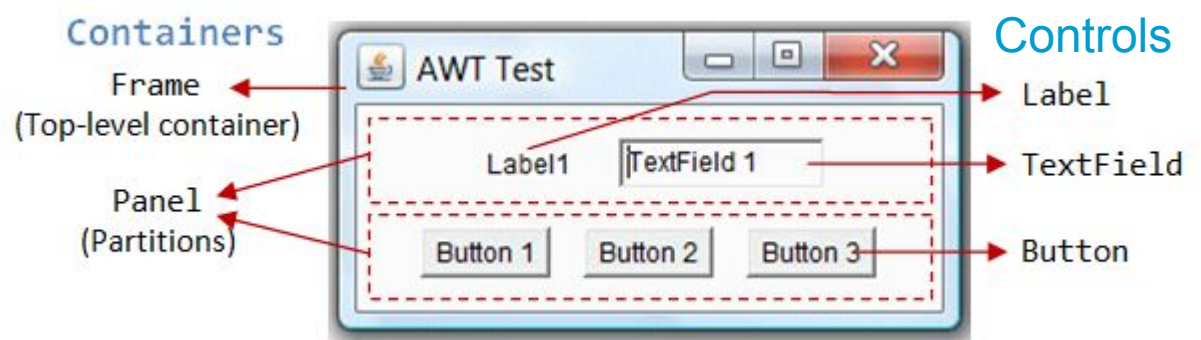
Install the **Chrome** or  
**Firefox extension**



Make sure you are in  
**presentation mode**

**wooclap**

# Componentes AWT



## Componentes

- Contenedores: contienen otros componentes

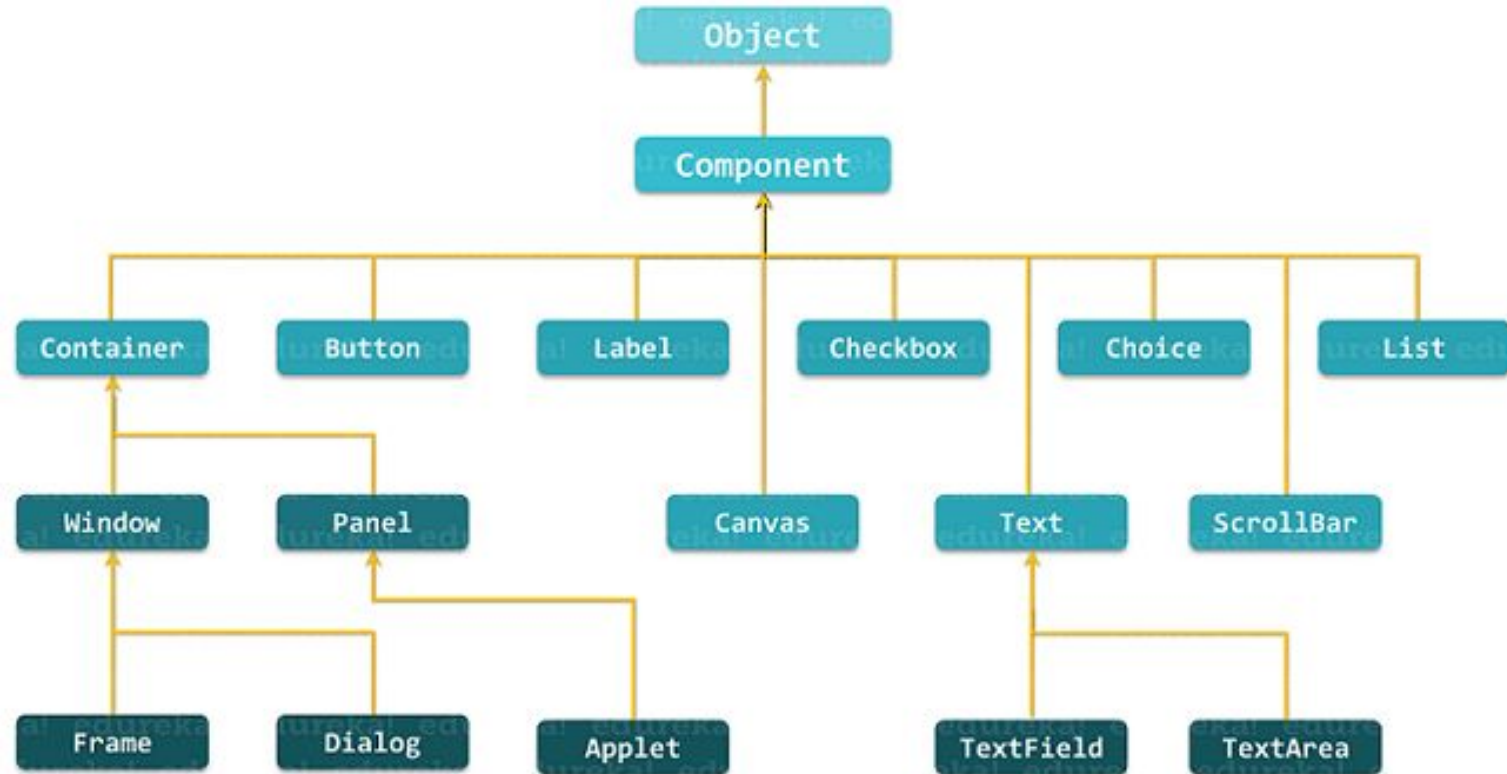
Window, Frame, Pane...

- Controles: realizan alguna acción

Button, TextField, Label...

Todos los elementos que se pueden ver se llaman componentes

# Jerarquía de los componentes



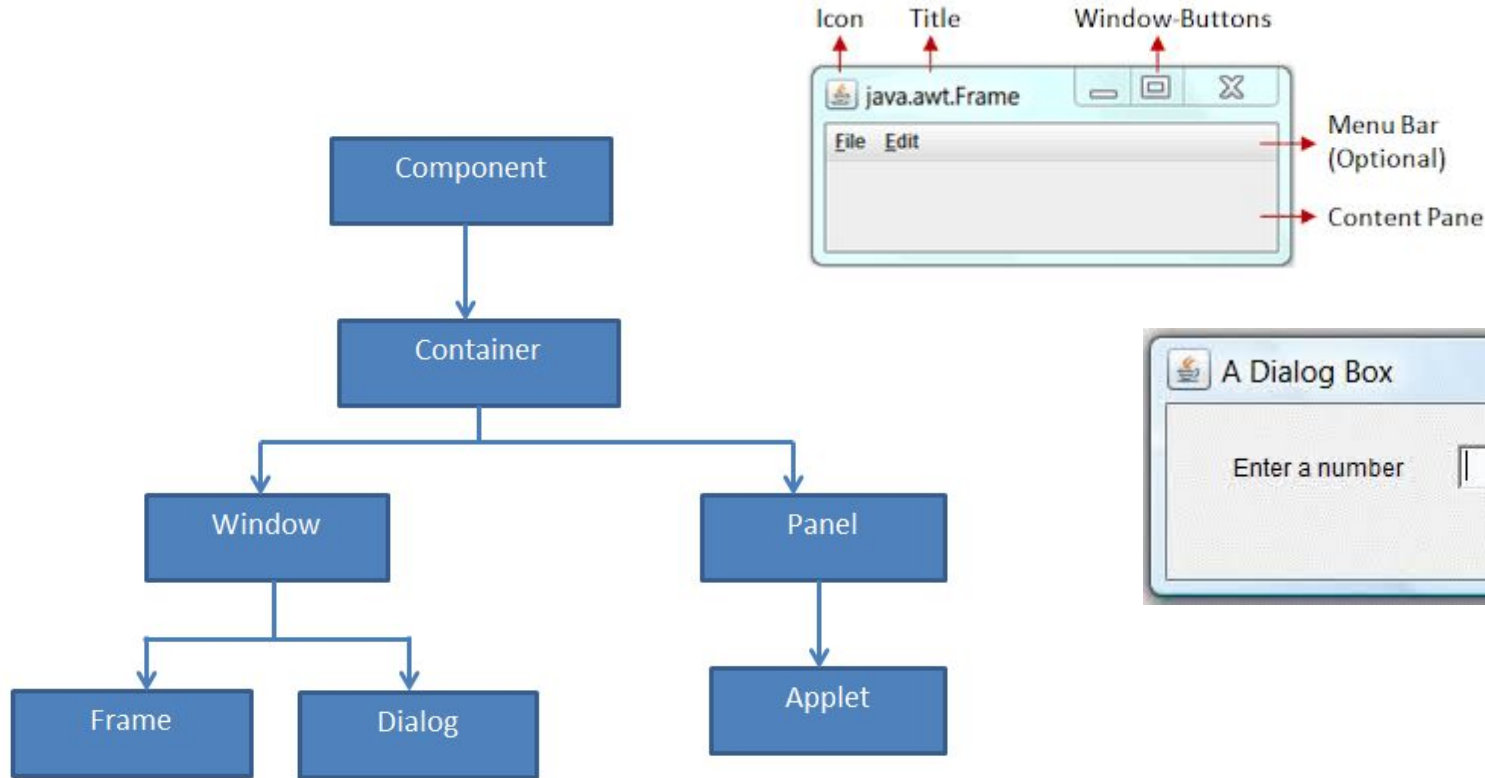


Fig : AWT Hierarchy



# Pasamos al código



```
Frame frame = new Frame("Hola mundo!");

frame.setBackground(Color.LIGHT_GRAY);
frame.setLayout(null);
frame.setSize(300, 300);
frame.setVisible(true); // Por defecto es false y no se muestra

// Control
Button button = new Button("Salir");
button.addActionListener(new ActionListener() {
    @Override
    public void actionPerformed(ActionEvent e) {
        ((Frame)button.getParent()).dispose();
    }
});

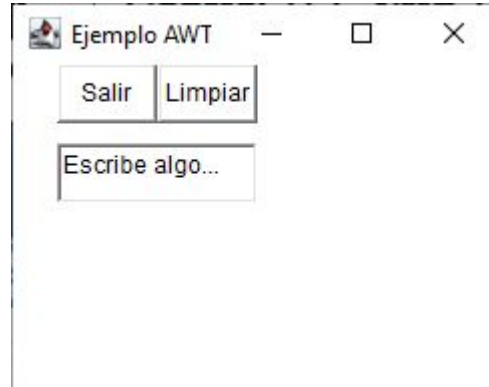
button.setBounds(150, 150, 50, 50);

// Añade el control al contenedor
frame.add(button);
```

# Ejercicio 1



Implementa una ventana como la siguiente



# Layout

null → Posicionamiento absoluto

LayoutManager

- BorderLayout
- CardLayout
- FlowLayout
- GridLayout

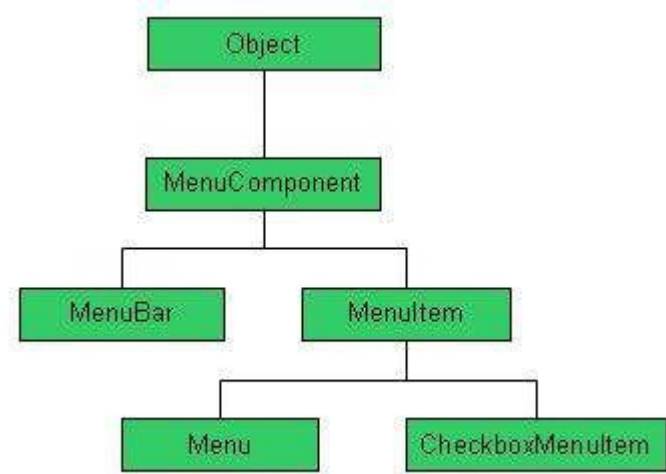


Ejemplos:

<https://www.oracle.com/technical-resources/articles/javase/awtlayoutmgr.html>

Los componentes del menú son subclases de MenuComponent:

- MenuBar
- Menu
- MenuItem
- MenuShortcut
- PopupMenu
- CheckboxMenuItem



Suceso que ocurre durante la ejecución del programa (como hacer clic en un botón, escribir en un campo o cerrar una ventana) y que el programa puede detectar y gestionar mediante código.

Nos van a permitir que la interfaz gráfica responda a las acciones del usuario.

Cuando el usuario realiza una acción, automáticamente se genera un objeto con información sobre esta acción.

# Your wooclap poll will be displayed here



Install the **Chrome** or  
**Firefox extension**



Make sure you are in  
**presentation mode**

**wooclap**

¿Quién los genera?

- Puede ser generado por el usuario (clic, teclado)
- Por el sistema (temporizador, cambio de estado)
- Por otro componente del programa

Ejemplos:

- El usuario hace clic en un botón → `ActionEvent`
- El cursor entra en un área de la ventana → `MouseEvent`
- Se cierra una ventana → `WindowEvent`
- El usuario presiona una tecla → `KeyEvent`

Un evento se compone de:

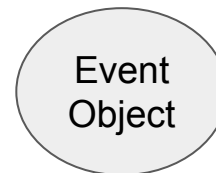
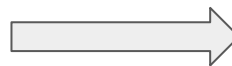
1. **Una fuente** ➤ El componente que genera el evento (por ejemplo, un botón).
2. **Un objeto evento** ➤ Contiene información sobre el suceso ocurrido.
3. **Un oyente (listener)** ➤ El código que se activa cuando ocurre el evento.

Listener es una interfaz que contiene métodos para manejar el evento. Hay que registrarla en la fuente para que esta la notifique.

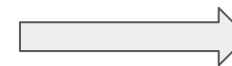
- Evento
  - Listener
  - Event handler
- Se registra un listener para cada tipo de evento que queramos atender
- Se sobrescriben/implementan los métodos que nos interesen en función de qué queramos hacer para responder al evento



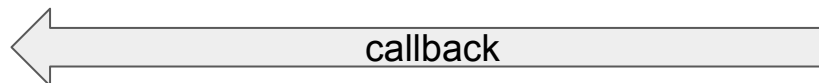
Se genera un objeto  
que representa al  
evento



Este es capturado y  
manejado por el  
listener



```
Listener {  
    Event handler {  
        ...  
    }  
}
```



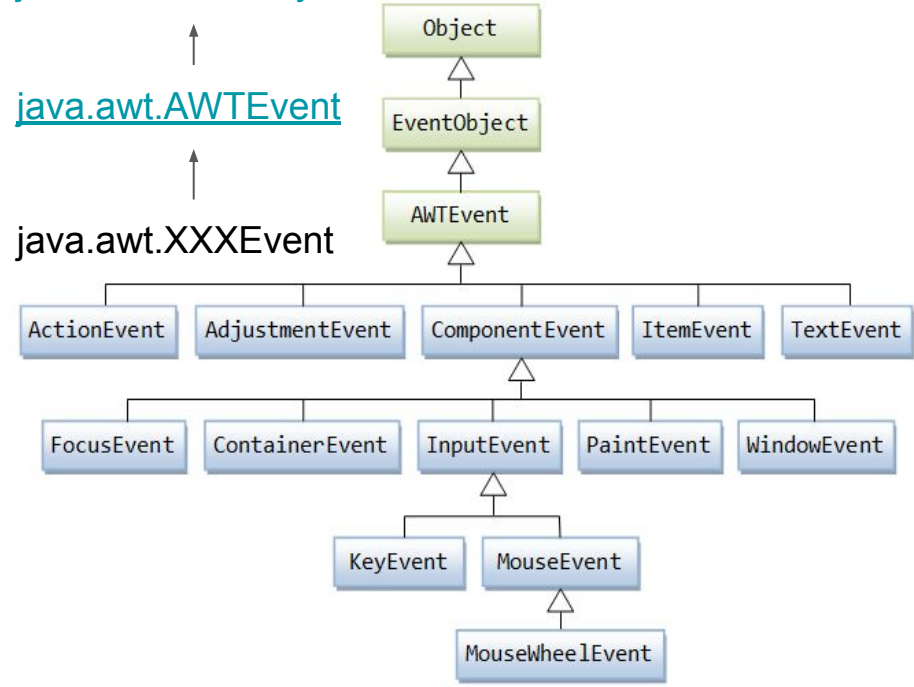


# Eventos: Clases e Inter

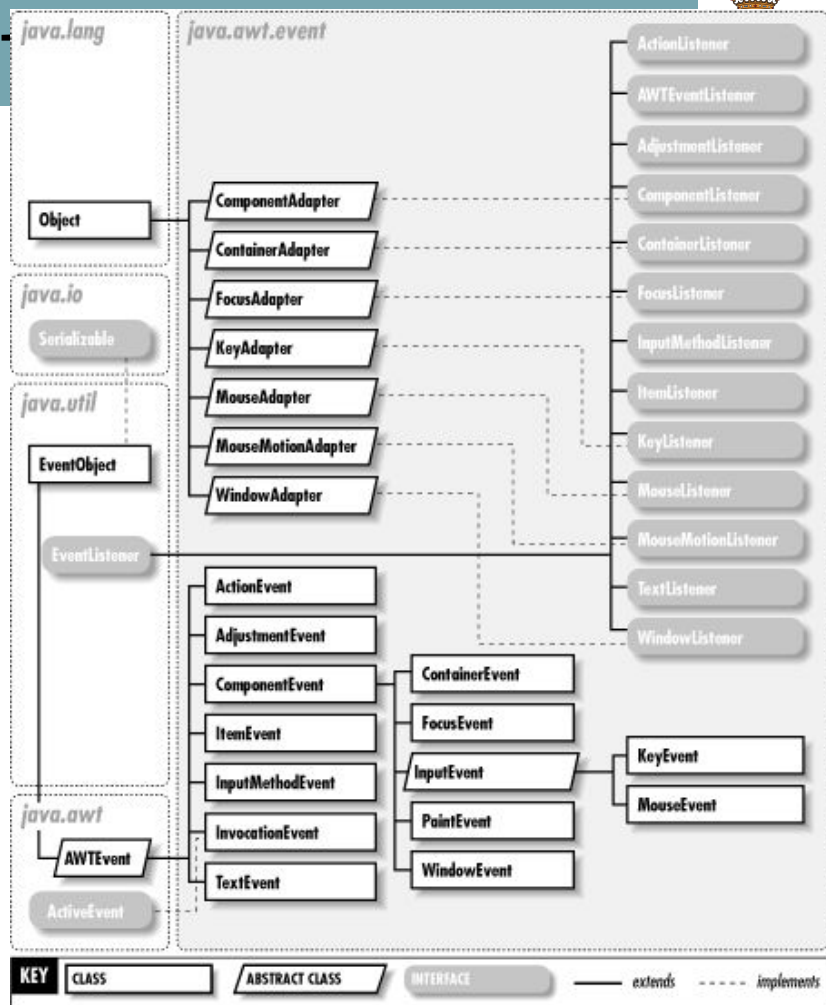
[java.util.EventObject](#)

[java.awt.AWTEvent](#)

java.awt.XXXEvent



Fuente: [www3.ntu.edu.sg](http://www3.ntu.edu.sg)



Fuente: [Java Foundation Classes in a nutshell. O'Reilly](#)

# Eventos: Clases e Interfaces

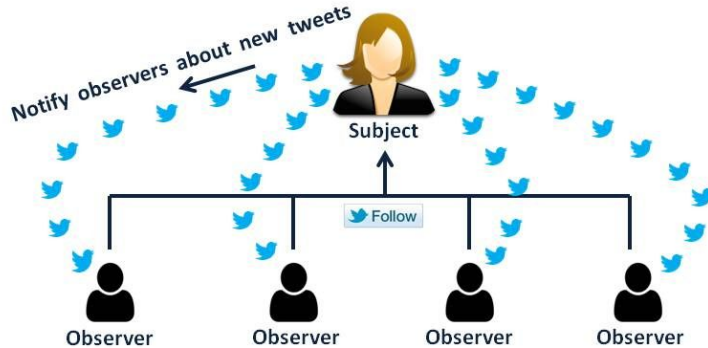


| Event class                 | Acciones sobre Componentes                                                                               | Listeners interfaces                                                           | Adapter       |
|-----------------------------|----------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------|---------------|
| <a href="#">ActionEvent</a> | Presionar un Button,<br>escribir en un TextField,<br>doble click en una List,<br>seleccionar un MenuItem | <a href="#">ActionListener</a><br><code>actionPerformed (ActionEvent e)</code> | -             |
| WindowEvent                 | Cualquier acción sobre el<br>Component Window                                                            | WindowListener                                                                 | WindowAdapter |
| MouseEvent                  | Mover, arrastrar, hacer<br>click... sobre un<br>Component                                                | MouseListener<br>MouseMotionListener                                           | MouseAdapter  |
| ...                         |                                                                                                          |                                                                                |               |

Los listeners que queramos se encarguen de manejar un evento, debemos registrarlos en el componente para que sean notificados por este cuando se genere el evento → Observer pattern.

Es como subscribirse a un canal de noticias

## Observer Design Pattern



# Evento: Registrar listeners



Para registrar un listener en un componente usamos el método:

```
someComponent.addXXXListener(instanceOfMyHandlerEventClass);
```

Para dar de baja el listener:

```
someComponent.removeXXXListener(instanceOfMyHandlerEventClass);
```

donde XXX es la clase de evento: Action, Component, Key, Window...

<https://docs.oracle.com/javase/tutorial/uiswing/events/actionlistener.html>

## Ejemplo

```
Button button = new Button("Salir");
button.addActionListener(new ActionListener() {
    @Override
    public void actionPerformed(ActionEvent e) {
        ((Frame)button.getParent()).dispose();
    }
});
```

# Ejemplo



```
import javax.swing.*;
import java.awt.event.*;

public class EjemploEvento extends JFrame implements ActionListener {
    JButton boton;

    public EjemploEvento() {
        boton = new JButton("Haz clic");
        boton.addActionListener(this); // Registrar el oyente
        add(boton);
        setSize(200, 200);
        setLayout(null);
        boton.setBounds(50, 80, 100, 30);
        setVisible(true);
        setDefaultCloseOperation(EXIT_ON_CLOSE);
    }

    @Override
    public void actionPerformed(ActionEvent e) {
        JOptionPane.showMessageDialog(this, "¡Botón pulsado!");
    }

    public static void main(String[] args) {
        new EjemploEvento();
    }
}
```

En la pantalla de login de la actividad anterior, incluye todos los eventos que consideres necesarios para que el usuario pueda:

- Cerrar la ventana
- El botón cancelar borre el texto introducido en todo el formulario
- El botón aceptar muestre un diálogo modal dando la bienvenida al usuario
- Opcional: permitir al usuario seleccionar un fichero y cuando lo haga mostrar la ruta y el nombre del fichero seleccionado.

# Your wooclap poll will be displayed here



Install the **Chrome** or  
**Firefox extension**



Make sure you are in  
**presentation mode**

**wooclap**

- Swing
- JavaFX

# Actividades finales



1º DAW  
Programación

UD 11: Swing

1. Práctica
2. Portfolio
3. Feedback